

Import Libraries

```
%%capture
%pip install watchdog
```

```
%%capture
%pip install pathlib
```

```
import os, shutil, time
from pathlib import Path
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
```

```
EXT_MAP = {
    ".pdf": "PDFs", ".jpg": "Images",
    ".png": "Images", ".csv": "Data",
    ".xlsx": "Data", ".mp4": "Videos",
}

def organise(folder: str):
    src = Path(folder)
    for f in src.iterdir():
        if f.is_file():
            dest = src / EXT_MAP.get(f.suffix, "Other")
            dest.mkdir(exist_ok=True)
            shutil.move(str(f), dest / f.name)
            print(f"Moved {f.name} → {dest.name}/")

organise("/content/sample_data")
```

```
Moved anscombe.json → Other/
Moved README.md → Other/
Moved california_housing_train.csv → Data/
Moved mnist_train_small.csv → Data/
Moved mnist_test.csv → Data/
Moved california_housing_test.csv → Data/
```

Section 2 — Web scraping automation

```
#Scrape a webpage on a schedule
#Parse content with BeautifulSoup
#Save results to CSV automatically
!pip install requests beautifulsoup4 -q
import requests, csv
from bs4 import BeautifulSoup
from datetime import datetime

def scrape_quotes() -> list:
    url = "https://quotes.toscrape.com"
    r = requests.get(url, timeout=10)
    soup = BeautifulSoup(r.text, "html.parser")
    quotes = []
    for q in soup.select(".quote")[:5]:
        quotes.append({
            "text": q.find("span", class_="text").text,
            "author": q.find("small").text,
            "scraped_at": datetime.now().isoformat(),
        })
    return quotes

def save_to_csv(rows, path="quotes.csv"):
    with open(path, "a", newline="") as f:
        w = csv.DictWriter(f, fieldnames=["text", "author", "scraped_at"])
        w.writerows(rows)

data = scrape_quotes()
save_to_csv(data)
print(f"Saved {len(data)} quotes")
```

Saved 5 quotes

SMTP email sender

```
import smtplib
from email.mime.text import MIMEText

def send_alert(subject: str, body: str,
               to: str, gmail_user: str, app_pw: str):
    msg = MIMEText(body)
    msg["Subject"] = subject
    msg["From"] = gmail_user
    msg["To"] = to
    with smtplib.SMTP_SSL("smtp.gmail.com", 465) as s:
        s.login(gmail_user, app_pw)
        s.send_message(msg)
    print("Alert sent ✓")
```

Usage example (use an App Password, not your real password!)

```
!pip install schedule -q
import schedule, time
```

```
# send_alert("Script done", "Scraping finished.", "you@gmail.com",
#           "bot@gmail.com", "abcd efgh ijkl mnop")

def job_scrape():
    data = scrape_quotes()
    save_to_csv(data)
    print(f"[{datetime.now():%H:%M:%S}] Scraped {len(data)} rows")

def job_report():
    print(f"[{datetime.now():%H:%M:%S}] Daily report sent")

# Every 10 seconds (for demo; change to .minutes / .hours in prod)
schedule.every(10).seconds.do(job_scrape)
# Every day at 08:00
schedule.every().day.at("08:00").do(job_report)

# Run for 35 seconds (3 ticks) then stop
deadline = time.time() + 35
while time.time() < deadline:
    schedule.run_pending()
    time.sleep(1)
```

```
[09:03:37] Scraped 5 rows
[09:03:48] Scraped 5 rows
[09:03:59] Scraped 5 rows
```

```
!pip install apscheduler -q
from apscheduler.schedulers.background import BackgroundScheduler
from apscheduler.triggers.cron import CronTrigger
import time
```

```
scheduler = BackgroundScheduler()

# Interval job - every 15 seconds
scheduler.add_job(job_scrape, "interval", seconds=15,
                  id="scraper")

# Cron job - weekdays at 07:30
scheduler.add_job(job_report,
                  CronTrigger(day_of_week="mon-fri", hour=7, minute=30),
                  id="daily_report")

scheduler.start()
print("Scheduler running. Jobs:")
for job in scheduler.get_jobs():
    print(f" • {job.id} - next: {job.next_run_time}")

time.sleep(40) # keep alive for demo
scheduler.shutdown()
```

```
Scheduler running. Jobs:
• scraper — next: 2026-06-04 09:04:29.790386+00:00
• daily_report — next: 2026-06-05 07:30:00+00:00
[09:04:30] Scraped 5 rows
[09:04:45] Scraped 5 rows
```

Argparse

```
import argparse

parser = argparse.ArgumentParser()

parser.add_argument(
    "--folder",
    required=True,
    help="Folder to organize"
)

args = parser.parse_args()

organise(args.folder)
```

Subprocess

```
import subprocess

def job_scrape():

    data = scrape_quotes()
    save_to_csv(data)

    subprocess.run([
        "python",
        "send_report.py"
    ])
```

Schedule

```
import schedule
import time

def send_daily_email():

    send_alert(
        subject="Daily Automation Report",
        body="Automation completed successfully.",
        to="receiver@gmail.com",
        gmail_user="bot@gmail.com",
        app_pw="app_password"
    )

schedule.every().day.at("18:00").do(
    send_daily_email
)

while True:
    schedule.run_pending()
    time.sleep(1)
```

APScheduler Cron

```
from apscheduler.schedulers.blocking import BlockingScheduler

scheduler = BlockingScheduler()

def report_job():
    print("Daily report generated")

scheduler.add_job(
    report_job,
    trigger='cron',
    hour=8
```

```
minute=0  
)  
  
scheduler.start()
```